```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.model_selection import train_test_split

tf.random.set_seed(42)
np.random.seed(42)

df=pd.read_csv("mnist_784_csv.csv")
print(df.shape)
```

```
(70000, 785)
```

```
x=df.iloc[:,:-1].values.astype("float32")/255.0

x=x.reshape(-1,28,28,1)
print(x.shape)
print(x.min())
print(x.max())
```

```
(70000, 28, 28, 1)
0.0
1.0
```

```
x_train,x_test=train_test_split(x,test_size=0.2, random_state=42)
print(x_train.shape)
print(x_test.shape)
```

```
(56000, 28, 28, 1)
(14000, 28, 28, 1)
```

```
noise_factor=0.5

x_train_noisy=x_train+noise_factor*np.random.normal(loc=0.0,scale=1.0,
size=x_train.shape)
x_test_noisy=x_test+noise_factor*np.random.normal(loc=0.0,scale=1.0,si
ze=x_test.shape)

x_train_noisy=np.clip(x_train_noisy,0.0,1.0).astype("float32")
x_test_noisy=np.clip(x_test_noisy,0.0,1.0).astype("float32")

n=10
plt.figure(figsize=(15, 3))
for i in range(n):
    plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
```

```
    if i == 0:
        plt.title("Clean", loc="left")
    plt.subplot(2, n, n + i + 1)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
    if i == 0:
        plt.title("Noisy", loc="left")
plt.show()
```




```
autoencoder = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),

    layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
    layers.UpSampling2D((2, 2)),
    layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
    layers.UpSampling2D((2, 2)),
    layers.Conv2D(1, (3, 3), activation="sigmoid", padding="same"),
])

autoencoder.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["mse"])
autoencoder.summary()
```

```
Model: "sequential_3"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
━━━┩
│ conv2d_15 (Conv2D)                   │ (None, 28, 28, 32)          │
320 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
───┤
```

```
| max_pooling2d_6 (MaxPooling2D)  | (None, 14, 14, 32)     |
0 |
├─────────────────────────────────┼────────────────────────┼──────
──────┤
| conv2d_16 (Conv2D)              | (None, 14, 14, 32)     |
9,248 |
├─────────────────────────────────┼────────────────────────┼──────
──────┤
| max_pooling2d_7 (MaxPooling2D)  | (None, 7, 7, 32)       |
0 |
├─────────────────────────────────┼────────────────────────┼──────
──────┤
| conv2d_17 (Conv2D)              | (None, 7, 7, 32)       |
9,248 |
├─────────────────────────────────┼────────────────────────┼──────
──────┤
| up_sampling2d_6 (UpSampling2D)  | (None, 14, 14, 32)     |
0 |
├─────────────────────────────────┼────────────────────────┼──────
──────┤
| conv2d_18 (Conv2D)              | (None, 14, 14, 32)     |
9,248 |
├─────────────────────────────────┼────────────────────────┼──────
──────┤
| up_sampling2d_7 (UpSampling2D)  | (None, 28, 28, 32)     |
0 |
├─────────────────────────────────┼────────────────────────┼──────
──────┤
| conv2d_19 (Conv2D)              | (None, 28, 28, 1)      |
289 |
└─────────────────────────────────┴────────────────────────┴──────
──────┘

 Total params: 28,353 (110.75 KB)

 Trainable params: 28,353 (110.75 KB)

 Non-trainable params: 0 (0.00 B)
```

```
history = autoencoder.fit(
    x_train_noisy, x_train,
    validation_split=0.1,
    epochs=10,
    batch_size=128,
)
```

```
Epoch 1/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 101s 250ms/step - loss: 0.1841 - mse:
0.0392 - val_loss: 0.1234 - val_mse: 0.0199
Epoch 2/10
```

```
394/394 ━━━━━━━━━━━━━━━━━━━━ 137s 237ms/step - loss: 0.1175 - mse:
0.0178 - val_loss: 0.1137 - val_mse: 0.0167
Epoch 3/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 93s 236ms/step - loss: 0.1114 - mse:
0.0159 - val_loss: 0.1097 - val_mse: 0.0154
Epoch 4/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 142s 238ms/step - loss: 0.1081 - mse:
0.0148 - val_loss: 0.1072 - val_mse: 0.0146
Epoch 5/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 94s 238ms/step - loss: 0.1058 - mse:
0.0141 - val_loss: 0.1053 - val_mse: 0.0140
Epoch 6/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 94s 237ms/step - loss: 0.1042 - mse:
0.0135 - val_loss: 0.1038 - val_mse: 0.0135
Epoch 7/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 142s 237ms/step - loss: 0.1029 - mse:
0.0131 - val_loss: 0.1025 - val_mse: 0.0130
Epoch 8/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 142s 236ms/step - loss: 0.1018 - mse:
0.0128 - val_loss: 0.1016 - val_mse: 0.0127
Epoch 9/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 142s 236ms/step - loss: 0.1010 - mse:
0.0125 - val_loss: 0.1008 - val_mse: 0.0125
Epoch 10/10
394/394 ━━━━━━━━━━━━━━━━━━━━ 93s 235ms/step - loss: 0.1003 - mse:
0.0123 - val_loss: 0.1002 - val_mse: 0.0123
```

```
denoised =autoencoder.predict(x_test_noisy)

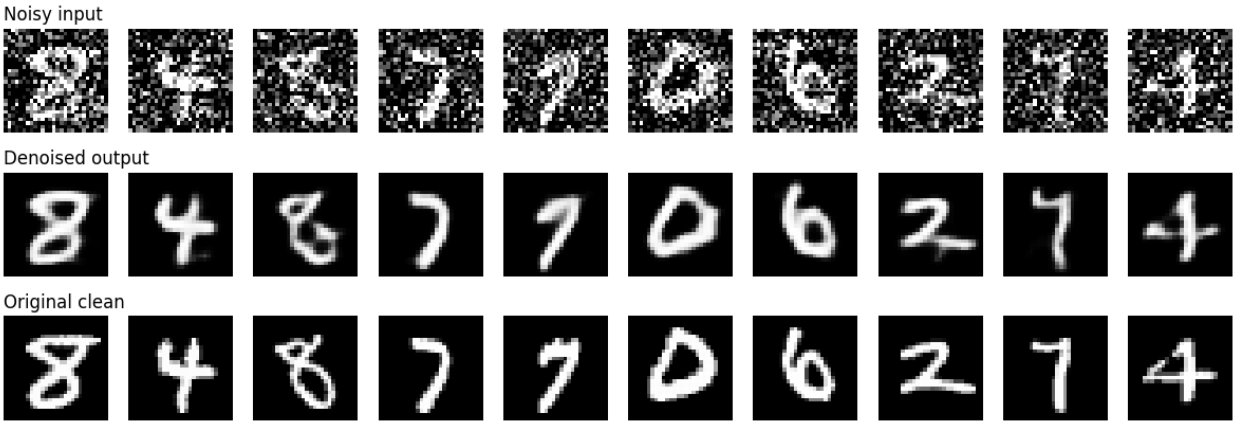
mse_noisy=np.mean((x_test_noisy-x_test)**2)
mse_denoised=np.mean((denoised-x_test)**2)

print("MSE noisy vs clean", mse_noisy)
print("MSE denoised vs clean", mse_denoised)

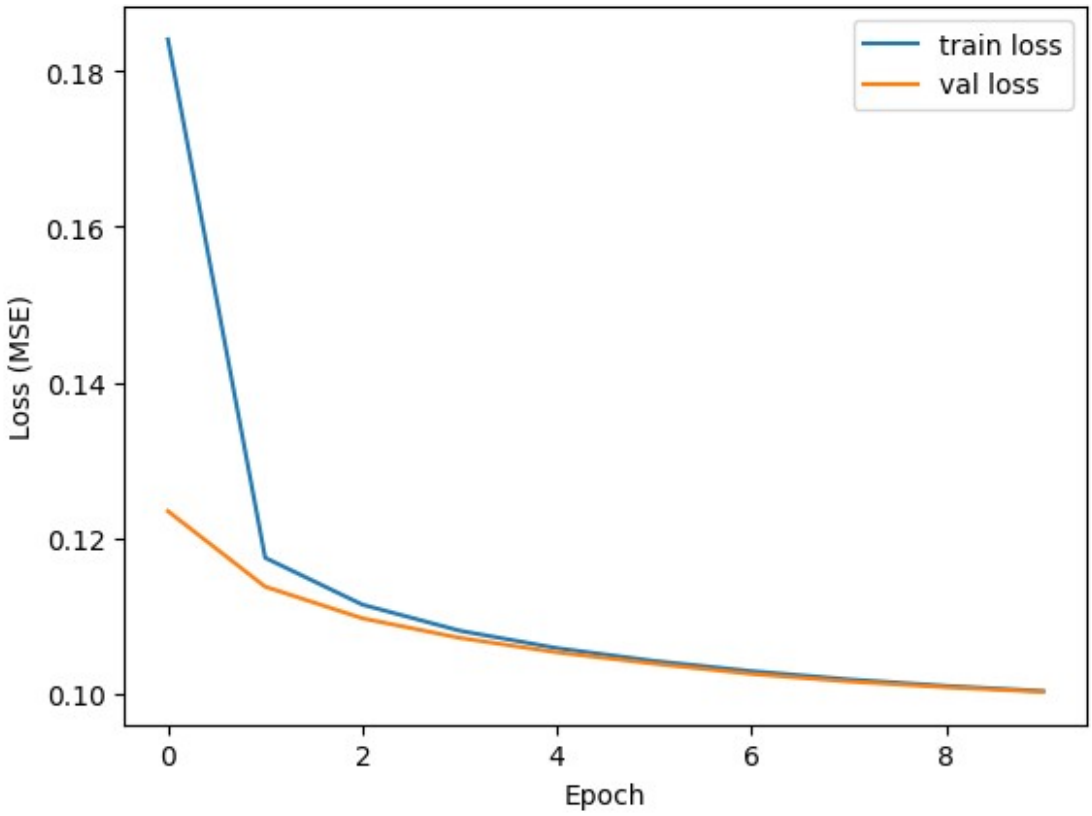
plt.figure(figsize=(15, 5))
for i in range(n):
    plt.subplot(3, n, i + 1)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
    if i == 0:
        plt.title("Noisy input", loc="left")
    plt.subplot(3, n, n + i + 1)
    plt.imshow(denoised[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
    if i == 0:
        plt.title("Denoised output", loc="left")
    plt.subplot(3, n, 2 * n + i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
```

```
    if i == 0:
        plt.title("Original clean", loc="left")
plt.show()
```

```
438/438 ━━━━━━━━━━━━━━━━━━━━ 8s 18ms/step
MSE noisy vs clean 0.115534745
MSE denoised vs clean 0.012244556
```




```
plt.plot(history.history["loss"], label="train loss")
plt.plot(history.history["val_loss"], label="val loss")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.legend()
plt.show()
```


train loss
val loss
0.18
0.16
0.14
0.12
0.10
Loss (MSE)
0
2
4
6
8
Epoch